

Sistemas Computacionais

Suporte à resolução dos
exercícios MIPS - 1, 2, 3 e 4

Aulas Práticas – MIPS

Nuno Veiga, Rui Vasco Monteiro

Exercício 1a

Elabore um programa em assembler (MIPS64) que implemente o seguinte código escrito em linguagem C:

a)

```
int c, a=10, b=8;  
c = a + b;
```

b)

```
int c, a=10, b=8, z=2;  
c = a + b + z;
```

Exercício 1a

1. Na área de trabalho do seu computador crie uma pasta denominada **“SC”**
2. Aceda ao moodle da UC e identifique o local onde se encontra o ficheiro **“ex1a-sum.s”**
3. Copie esse ficheiro para a pasta **“SC”**:
 - Confirme que o ficheiro tem a extensão **“.s”**
4. Utilizando o **notepad ++** abra o ficheiro **“ex1a-sum.s”** e confirme o seu conteúdo....
5. Aceda novamente ao moodle da UC e descarregue o ficheiro **“winmips64.zip”** para a pasta **“SC”**. Descompacte esse ficheiro...
6. Execute o ficheiro **“winmips64.exe”** e abra agora o ficheiro **“ex1a-sum.s”**
7. Para executar o código que se encontra no ficheiro **“ex1-sum.s”**, aceda ao menu **“Execute”** e escolha ...
 - **“Single Cycle”** ou prima a **tecla F7** – para execução ciclo a ciclo
 - **“Run to”** ou prima a **tecla F4** – executa o código de uma só vez
8. Tente compreender o conteúdo das várias janelas....

```
; sum example
```

```
.data
```

```
A: .word 10
```

```
B: .word 8
```

```
C: .word 0
```

```
.text
```

```
main:
```

```
ld r4,A(r0)
```

```
ld r5,B(r0)
```

```
dadd r3,r4,r5
```

```
sd r3,C(r0)
```

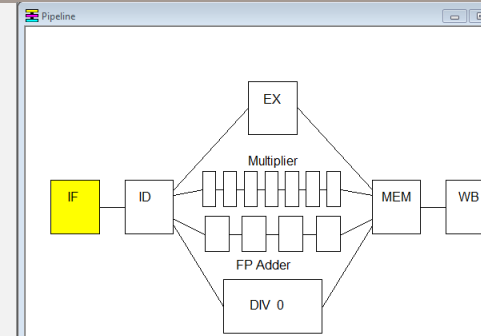
```
halt
```

Exercício 1a – WINMIPS64

• Algumas janelas:

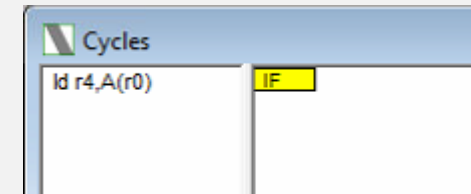
○ **Pipeline:**

- Esquema visual do *pipeline*, e da fase em que se encontra cada instrução



○ **Cyles:**

- Disposição temporal do estado da instrução dentro do *pipeline*



○ **Statistics:**

- Informação estatística, útil para otimização de código

```

Statistics
Execution
10 Cycles
5 Instructions
2.000 Cycles Per Instruction (CPI)

Stalls
1 RAW Stall
0 WAW Stalls
0 WAR Stalls
0 Structural Stalls
0 Branch Taken Stalls
0 Branch Misprediction Stalls

Code size
20 Bytes
    
```

○ **Registers (CPU):**

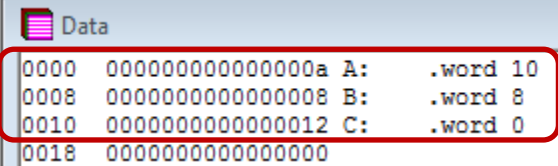
- Contém os valores dos **Registos GPR** (*General Purpose Registers*)
- Cada registo GPR ocupa 8 B (64 bits), representado por 16 caracteres hexadecimais (16 x 4 bits = 64 bits)
- Registo **R0 = 0**

```

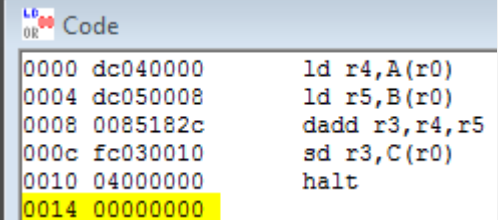
Registers
R0= 0000000000000000
R1= 0000000000000000
R2= 0000000000000000
R3= 0000000000000012
R4= 000000000000000a
R5= 0000000000000008
R6= 0000000000000000
    
```

Exercício 1a – WINMIPS64

- Algumas janelas (continuação):
 - Data (Memória de Dados ou Cache L1 de Dados):**
 - Permite observar o estado da memória de dados:
 - Tamanho ocupado pelas variáveis declaradas
 - Valores iniciais e valores atuais das variáveis
 - Contém **endereço de memória** + **conteúdo** armazenado em cada posição de mem.
 - Cada posição ocupa 64 bits (16 carateres hexadecimais = 16 x 4 bits), por isso o endereço de memória é incrementado de 8 B
 - Code (Memória de Instruções ou Cache L1 de Instruções):**
 - Permite observar o estado da memória de instruções (área de código)
 - Contém **endereço de cada instrução** (cada instrução ocupa 4 B/ 32 bits) + **codificação de cada instrução em código Máquina** (8 carateres hexadecimais = 8 x 4 bits = 32 bits ou 4 B) + **Código Assembler**
- Em qualquer altura pode-se carregar qualquer outro novo código e iniciar a sua execução, fazendo o seguinte conjunto de ações:
 - Aceda a **“File + Open”** ou execute a combinação de teclas **“CTRL + O”**
 - Prima a **tecla F7** ou **tecla F4**



Address	Hex Value	Variable	Size
0000	0000000000000000a	A:	.word 10
0008	00000000000000008	B:	.word 8
0010	000000000000000012	C:	.word 0
0018	00000000000000000		



Address	Hex Value	Assembly Instruction
0000	dc040000	ld r4,A(r0)
0004	dc050008	ld r5,B(r0)
0008	0085182c	dadd r3,r4,r5
000c	fc030010	sd r3,C(r0)
0010	04000000	halt
0014	00000000	

Exercício 1a – WINMIPS64

- Podem-se aplicar os seguintes tipos de **Resets**:
 - Reset MIPS64** (premindo a combinação de teclas “**CTRL + R**” ou acedendo a “**File+ Reset MIPS64**”):

- Volta ao início do código, mas não apaga a memória

```

Code
0000 dc040000    ld r4,A(r0)
0004 dc050008    ld r5,B(r0)
0008 0085182c    dadd r3,r4,r5
000c fc030010    sd r3,C(r0)
0010 04000000    halt
  
```

```

Data
0000 0000000000000000a A:    .word 10
0008 0000000000000008 B:    .word 8
0010 0000000000000012 C:    .word 0
0018 0000000000000000
  
```

```

Registers
R0= 0000000000000000
R1= 0000000000000000
R2= 0000000000000000
R3= 0000000000000000
R4= 0000000000000000
R5= 0000000000000000
R6= 0000000000000000
  
```

- Reload** (premindo a tecla **F10** ou acedendo a “**File+ Reload**”)

- Volta ao início do código e apaga a memória

```

Code
0000 dc040000    ld r4,A(r0)
0004 dc050008    ld r5,B(r0)
0008 0085182c    dadd r3,r4,r5
000c fc030010    sd r3,C(r0)
0010 04000000    halt
  
```

```

Data
0000 0000000000000000a A:    .word 10
0008 0000000000000008 B:    .word 8
0010 0000000000000000 C:    .word 0
  
```

```

Registers
R0= 0000000000000000
R1= 0000000000000000
R2= 0000000000000000
R3= 0000000000000000
R4= 0000000000000000
R5= 0000000000000000
R6= 0000000000000000
  
```

- Full Reset** (premindo a combinação de teclas “**CTRL+F**” ou acedendo a “**File+ Full Reset**”):

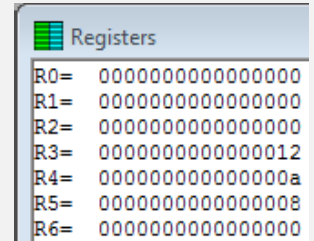
- Para voltar a executar o mesmo código:
 - Premir F10** (carregar o programa em memória)
 - Premir F7 ou F4**

```

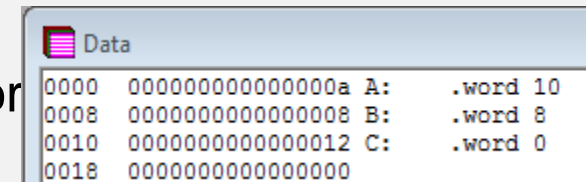
Data
0000 0000000000000000
0008 0000000000000000
0010 0000000000000000
  
```

MIPS – Registos e Variáveis MIPS64

- Como é que o MIPS guarda os dados a processar e/ou já processados?
 - **Registos (R0, R1, ..., R31) de Uso Geral (GPR)** que estão na CPU:
 - São as unidades mais rápidas para guardar dados
 - Têm um tamanho de 64 bits
 - Alimentam as Unidades de Execução (por ex. ALU) e guardam temporariamente os resultados da execução das Unidades de Execução
 - São “*not case sensitive*”, pelo que “**r0**” = “**R0**”
 - **Variáveis (A, B, C,...):**
 - São as variáveis em memória do computador
 - São *case sensitive*, pelo que “**A**” ≠ “**a**”
- Como é que o MIPS **movimenta os dados** entre **Registos** e **Memória**?
 - Da **Memória** para os **Registos** é utilizada a instrução “**load**”
 - Dos **Registos** para a **Memória** é utilizada a instrução “**store**”



Registers	
R0=	0000000000000000
R1=	0000000000000000
R2=	0000000000000000
R3=	0000000000000012
R4=	000000000000000a
R5=	0000000000000008
R6=	0000000000000000



Data		
0000	000000000000000a	A: .word 10
0008	0000000000000008	B: .word 8
0010	0000000000000012	C: .word 0
0018	0000000000000000	

Exercício 1a – Tamanho dos Dados, Instruções e Diretivas MIPS64

- Vamos agora analisar o código do ficheiro “**ex1a-sum.s**”...

Legenda:

```

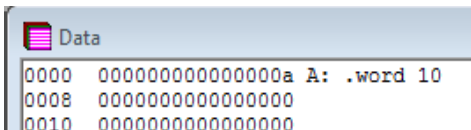
; sum example
.
.
.data
A: .word 10
B: .word 8
C: .word 0

.text
main:
ld r4,A(r0)
ld r5,B(r0)
dadd r3,r4,r5
sd r3,C(r0)

halt

```

- Comentários começam por “;”
- “.” indica que estamos perante uma diretiva do assembler
- Diretiva que indica o início do **segmento de dados**:
 - Permite declarar variáveis, as quais surgem na janela “Data”
 - É **case sensitive**
- Variável “A” declarada como do tipo **word** (ocupa **64 bits**) e possui o valor inicial de “10”



- Para mais informação sobre os tipos de dados manipulados pelo MIPS64, consultar o ficheiro “**SC - Tamanho dos dados.pdf**”...

Inteiros

Arquitetura	MIPS32	MIPS64
R1	32	64
.word	32	64
.word32	---	32
.word16	---	16
.half	16	---
.byte	8	

Exercício 1a – Tamanho dos Dados, Instruções e Diretivas MIPS64

```

; sum example
.data
A: .word 10
B: .word 8
C: .word 0

.text
main:
ld r4,A(r0)
ld r5,B(r0)

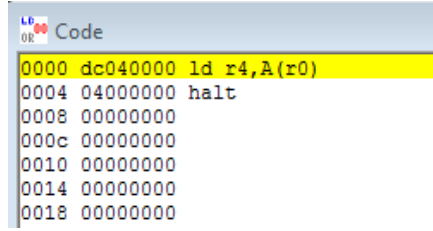
dadd r3,r4,r5

sd r3,C(r0)

halt
        
```

Legenda:

- Diretiva “**.text**” que indica o início do segmento de código:
 - As instruções surgem na janela “**Code**”
 - Também se pode utilizar a diretiva “**.code**”
- Etiqueta “**main:**”, a qual, neste caso, pode ser omitida
- Instrução **load double** (**registo**, **memória**):
 - Transfere (o numero “**10**”) da **memória de dados (variável A)** para o **registo R4**
 - São carregados 64 bits de dados (**double**)
 - Todas as instruções são “**not case sensitive**”, pelo que “**ld**” = “**LD**”
- Instrução **double add**:
 - Soma o conteúdo do registo R4 com R5 e coloca-o em R3 (**r3=r4+r5**)
- Instrução **store double** (**registo**, **memória**):
 - Transfere (o numero “**18**”) do **registo de 64 bits (R3)** para a **memória de dados (variável C)**
- Diretiva que permite **terminar a execução do código o programa** e devolver o controlo ao Sistema Operativo



Code

```

0000 dc040000 ld r4,A(r0)
0004 04000000 halt
0008 00000000
000c 00000000
0010 00000000
0014 00000000
0018 00000000
        
```

- Para mais informação sobre instruções e diretivas MIPS64, consultar o ficheiro “**SC-TabelaMIPS64 v1.pdf**”...

Load (ld, lw, lb)
d=64 bits
w=32 bits
b=8 bits

Store (sd, sw, sb)

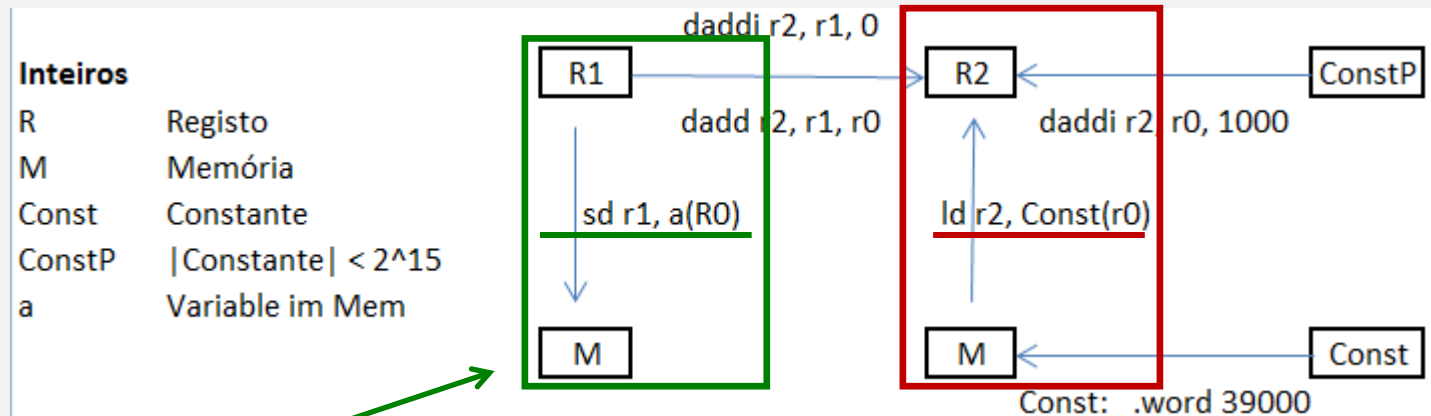
Movimentação de Dados entre Memória - Registos

```
.data
A: .word 10
B: .word 8
C: .word 0

.text
main:
    ld r4,A(r0)
    ld r5,B(r0)
    dadd r3,r4,r5
    sd r3,C(r0)

halt
```

- Carregar valor da **Memória (variável)** para **Registo**



- Carregar valor do **Registo** para a **Memória**

Nota: Para mais informação sobre movimentação de dados em MIPS64, consultar o ficheiro "**SC - MIPS - Movimentação de dados.pdf**"

Movimentação de Dados entre Memória - Registos

- Colocar uma **Constante** de “**valor pequeno**” num **Registo**:

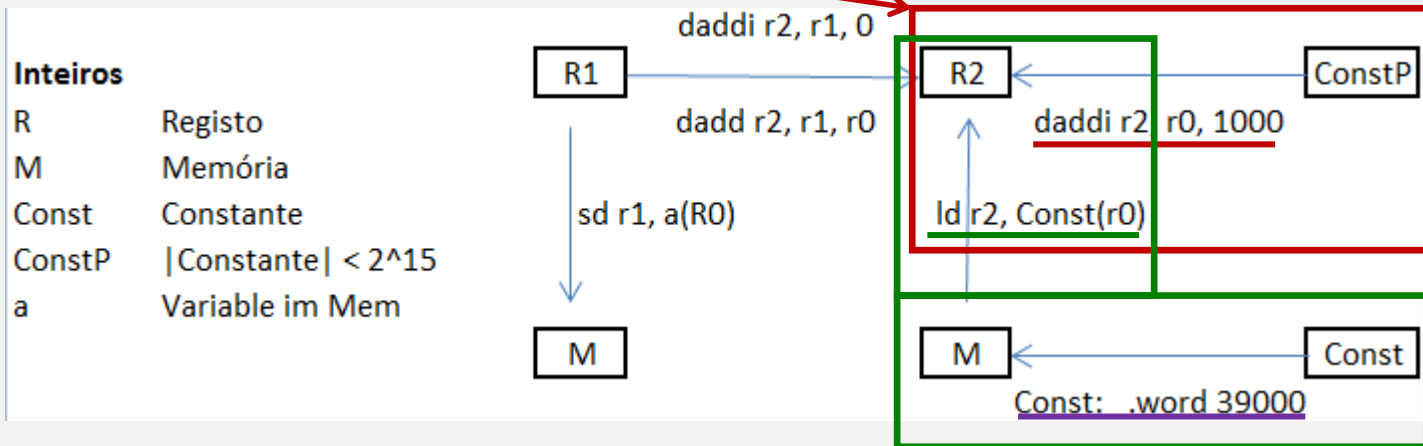
```
.data

.code
daddi r2,r0,1000
halt
```

Legenda:

➤ Instrução **double add immediate** :

- Soma o conteúdo do registo R0 ao valor da constante (1000) e coloca-o em R2



- Colocar **qualquer Constante** num **Registo**:

Nota: Para mais informação sobre movimentação de dados em MIPS64, consultar o ficheiro “**SC - MIPS - Movimentação de dados.pdf**”

Para mais informação sobre instruções e diretivas MIPS64, consultar o ficheiro “**SC -TabelaMIPS64 v1.pdf**”...

```
.data
Const:    .word 39000
.code
ld r2,Const(r0)
halt
```

Exercício 1b

Elabore um programa em assembler (MIPS64) que implemente o seguinte código escrito em linguagem C:

a)

```
int c, a=10, b=8;  
c = a + b;
```

b)

```
int c, a=10, b=8, z=2;  
c = a + b + z;
```

Exercício 1b

1. Utilizando o **notepad ++** abra novamente o ficheiro “**ex1a-sum.s**”
2. Aceda a “**Ficheiro + Guardar Como**” e grave o ficheiro com o nome “**ex1b.s**”
3. Faça as alterações necessárias para que seja possível somar 3 números, fazendo igualmente o carregamento do valor do **registo** para a **memória (variável)**:
 - **Carregar 3 valores da memória (variáveis A=10, B=8 e Z=2)** para **3 registos (r4, r5 e r6)**
 - **Somar esses 3 números e colocar o resultado em r3**
 - Transferir o resultado da soma (**20**) que está no **registo r3** para a **memória (variável C)**

```
; sum example
```

```
.data
```

```
A: .word 10
```

```
B: .word 8
```

```
Z: .word 2
```

```
C: .word 0
```

```
.text
```

```
main:
```

```
ld r4,A(r0)
```

```
ld r5,B(r0)
```

```
ld r6,Z(r0)
```

```
dadd r3,r4,r5
```

```
dadd r3,r3,r6
```

```
sd r3,C(r0)
```

```
halt
```

Exercício 2a

Elabore um programa em assembler (MIPS64) que troque o valor de 2 variáveis definidas em memória:

- a) Utilizando apenas referências de memória
- b) Efetuando a cópia para os registos e efetuando a troca através dos mesmos

Exercício 2a

1. Utilizando o **notepad ++** abra novamente o ficheiro “**ex1a-sum.s**”
2. Aceda a “**Ficheiro + Guardar Como**” e grave o ficheiro com o nome “**ex2a.s**”
3. **Elimine o que está indicado na figura ao lado**
4. Substitua a diretiva “**.text**” por “**.code**”
5. O código terá agora este aspeto:

```

.data
A: .word 10
B: .word 8

.code

halt

```

```

: sum example

.data
A: .word 10
B: .word 8
C: .word 0

.text
.code

main:
ld r4,A(r0)
ld r5,B(r0)
dadd r3,r4,r5
sd r3,C(r0)

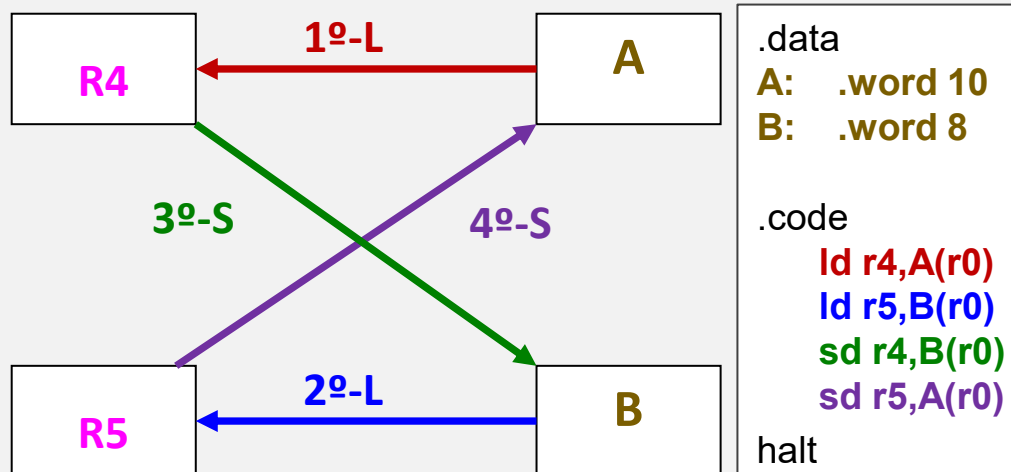
halt

```

6. Execute o código anterior no MIPS para confirmar que não há erros...

Exercício 2a

- A troca não pode ser efetuada diretamente entre **variáveis envolvidas** (“A” e “B”), pelo que temos de usar **registos** (R4 e R5)
- Algoritmo:
 - Carregar o conteúdo das **variáveis** para os **registos**:
 - Carregar o conteúdo da **variável A** para R4 (instrução “load” » 1º-L)
 - Carregar o conteúdo da **variável B** para R5 (instrução “load” » 2º-L)
 - Faz-se a troca de conteúdo diretamente para a **variável**:
 - Transferindo o conteúdo de R4 para B (instrução “store” » 3º-S)
 - Transferindo o conteúdo de R5 para A (instrução “store” » 4º-S)



- No início:

Data		
0000	000000000000000000a	A: .word 10
0008	0000000000000000008	B: .word 8
0010	0000000000000000000	
0018	0000000000000000000	

- No final:

Data		
0000	0000000000000000008	A: .word 10
0008	000000000000000000a	B: .word 8
0010	0000000000000000000	
0018	0000000000000000000	

Exercício 2b

Elabore um programa em assembler (MIPS64) que troque o valor de 2 variáveis definidas em memória:

- a) Utilizando apenas referências de memória
- b) Efetuando a cópia para os registos e efetuando a troca através dos mesmos

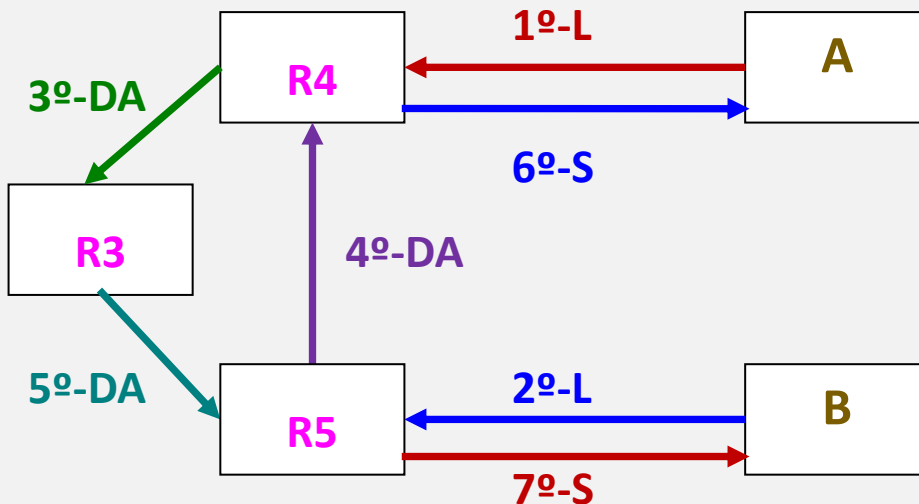
Exercício 2b

- Após carregar “A” e “B” para os **registos R4** e **R5**, utilizamos um **registo auxiliar (R3)** para efetuar a troca:
 - Carregar o conteúdo das variáveis para os registos:
 - Carregar o conteúdo da **variável A** para **R4** (instrução “load” » 1º-L)
 - Carregar o conteúdo da **variável B** para **R5** (instrução “load” » 2º-L)
 - Efetua-se a troca dos conteúdos dos **registos R4** e **R5**, usando o **registo R3**:
 - Colocar o conteúdo de **R4** em **R3** (instrução “dadd” » 3º-DA)
 - Colocar o conteúdo de **R5** em **R4** (instrução “dadd” » 4º-DA)
 - Colocar o conteúdo de **R3** em **R5** (instrução “dadd” » 5º-DA)
 - Colocar o resultado da troca nas **variáveis “A”** e **“B”**:
 - Transferindo o conteúdo de **R4** para **B** (instrução “store” » 6º-S)
 - Transferindo o conteúdo de **R5** para **A** (instrução “store” » 7º-S)

```
.data
A: .word 10
B: .word 8

.code
ld r4,A(r0)
ld r5,B(r0)
dadd r3,r4,r0
dadd r4,r5,r0
dadd r5,r3,r0
sd r4,A(r0)
sd r5,B(r0)

halt
```



Registers	
R0=	0000000000000000
R1=	0000000000000000
R2=	0000000000000000
R3=	000000000000000a
R4=	000000000000000a
R5=	0000000000000008

Registers	
R0=	0000000000000000
R1=	0000000000000000
R2=	0000000000000000
R3=	000000000000000a
R4=	0000000000000008
R5=	0000000000000008

Registers	
R0=	0000000000000000
R1=	0000000000000000
R2=	0000000000000000
R3=	000000000000000a
R4=	0000000000000008
R5=	000000000000000a

Exercício 3

Adapte o exercício 1 de forma a enviar o resultado para o terminal.

MIPS (MIPS64) – Assembler

- Mostrar valor inteiro com sinal

- Memory Mapped I/O Area:

Memory Mapped I/O area

Addresses of CONTROL and DATA registers

CONTROL: .word32 0x10000

DATA: .word32 0x10008

Set CONTROL = 1, Set DATA to Unsigned Integer to be output

Set CONTROL = 2, Set DATA to Signed Integer to be output

Set CONTROL = 3, Set DATA to Floating Point to be output

Set CONTROL = 4, Set DATA to address of string to be output

Set CONTROL = 5, Set DATA+5 to x coordinate, DATA+4 to y coordinate, and DATA to RGB colour to be output

Set CONTROL = 6, Clears the terminal screen

Set CONTROL = 7, Clears the graphics screen

Set CONTROL = 8, read the DATA (either an integer or a floating-point) from the keyboard

Set CONTROL = 9, read one byte from DATA, no character echo.

- Para **mostrar o resultado** da execução do código na janela “terminal” (enviar para o écran), em formato de inteiro com sinal, utiliza-se o código 2

Nota: Para mais informação sobre mapeamento de memória para E/S em MIPS64, consultar o ficheiro “**SC - MIPS64SimulatorIO.pdf**”

MIPS (MIPS64) - Assembler

• Memory Mapped I/O Area:

- Exemplo para mostrar o valor inteiro (“37”) com sinal (CODE 2) na janela “terminal” (enviar para écran)

```

.data
CR: .word32 0x10000 ; Control Register tem o endereço 10000 hex
DR: .word32 0x10008 ; Data Register tem o endereço 10008 hex
                        ; 0x é para indicar que é numero hexadecimal

.code
lwu r1, CR(r0)      ; end. mem. do Control Register em r1
lwu r2, DR(r0)      ; end. mem. do Data Register em r2

daddi r4, r0, 37    ; valor 37 (que é p/ ser mostrado na janela "terminal") é colocado em r4

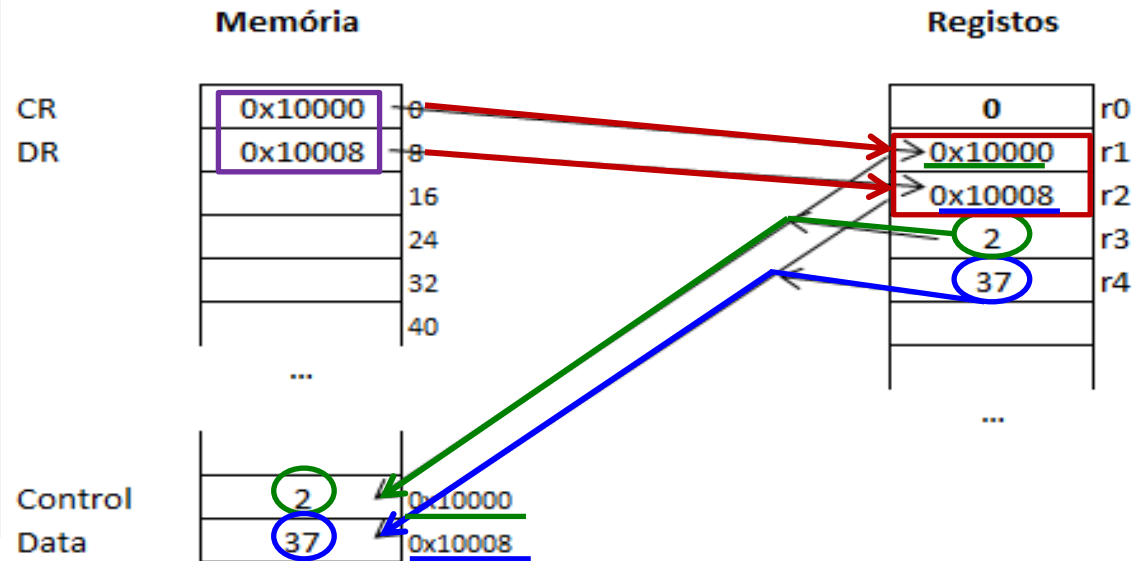
daddi r3, r0, 2     ; código 2 (código para mostrar inteiro com sinal) é colocado em r3

sd r4, (r2)         ; valor a mostrar é colocado no Data Register (envia o valor 37 de r4
                    ; para o endereço de memória 10008 hex)

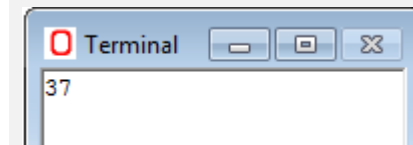
sd r3, (r1)         ; chama rotina p/ mostrar valor (envia o código 2 de r3 para o end. de
                    ; memória 10000 hex - Control Register)

halt
  
```

Signed Integer output to screen (Mostrar r4)



Set CONTROL = 2, Set DATA to Signed Integer to be output



MIPS (MIPS64) – Assembler

- Mostrar valor inteiro sem sinal

- Memory Mapped I/O Area:

Memory Mapped I/O area

Addresses of CONTROL and DATA registers

CONTROL: .word32 0x10000

DATA: .word32 0x10008

Set CONTROL = 1, Set DATA to Unsigned Integer to be output
 Set CONTROL = 2, Set DATA to Signed Integer to be output
 Set CONTROL = 3, Set DATA to Floating Point to be output
 Set CONTROL = 4, Set DATA to address of string to be output
 Set CONTROL = 5, Set DATA+5 to x coordinate, DATA+4 to y coordinate, and DATA to RGB colour to be output
 Set CONTROL = 6, Clears the terminal screen
 Set CONTROL = 7, Clears the graphics screen
 Set CONTROL = 8, read the DATA (either an integer or a floating-point) from the keyboard
 Set CONTROL = 9, read one byte from DATA, no character echo.

- Para **mostrar o resultado** da execução do código na janela “terminal” (enviar para o écran), em formato **de inteiro sem sinal**, utiliza-se o **código 1**

Nota: Para mais informação sobre mapeamento de memória para E/S em MIPS64, consultar o ficheiro “**SC - MIPS64SimulatorIO.pdf**”

MIPS (MIPS64) - Assembler

- *Memory Mapped I/O Area:*
 - Exemplo para mostrar o valor inteiro (“20”) sem sinal (**CODE 1**) na janela “terminal” (enviar para écran)

```

.data
CR: .word32 0x10000 ; Control Register tem o endereço 10000 hex
DR: .word32 0x10008 ; Data Register tem o endereço 10008 hex
    ; 0x é para indicar que é numero hexadecimal
.code
lwu r1, CR(r0)      ; end. mem. do Control Register em r1
lwu r2, DR(r0)      ; end. mem. do Data Register      em r2

daddi r4,r0,20       ; valor 20 (que é p/ ser mostrado na janela "terminal") é colocado em r4

daddi r3,r0,1        ; código 1(código para mostrar inteiro sem sinal) é colocado em r3

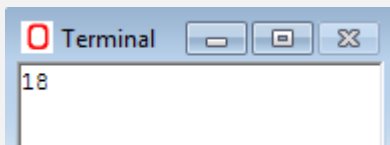
sd r4,(r2)           ; valor a mostrar é colocado no Data Register (envia o valor 20 de r4
    ; para o endereço de memória 10008 hex)

sd r3,(r1)           ; chama rotina p/ mostrar valor (envia o código 1 de r3 para o end. de
    ; memória 10000 hex - Control Register)
halt
  
```

Set CONTROL = 1 Set DATA to Unsigned Integer to be output

Exercício 3a – Resultado da Soma de 2 números enviado para o terminal

1. Utilizando o **notepad** ++ abra novamente o ficheiro “**ex1a-sum.s**”
2. Aceda a “**Ficheiro + Guardar Como**” e grave o ficheiro com o nome “**ex3a.s**”
3. Faça as alterações necessárias por forma a enviar o resultado para o terminal ...



```
.data
A:      .word 10
B:      .word 8
C:      .word 0
CR:      .word32 0x10000 ; Control Register tem o endereço 10000
DR:      .word32 0x10008 ; Data Register tem o endereço 10008

.code
lwu r1,CR(r0) ; end. mem. do Control Register em r1
lwu r2,DR(r0) ; end. mem. do Data Register em r2

ld r4,A(r0)
ld r5,B(r0)
dadd r3,r4,r5
sd r3,C(r0) ; Colocar a soma que está em r3 (18) na variável C
sd r3,(r2) ; Valor da soma a mostrar é colocado no Data Register
;(envia soma de r3 p/ end. de memória 10008 hex)

daddi r10,r0,1 ; Código 1 (código p/ mostrar inteiro sem sinal)
; colocado em r10
sd r10,(r1) ; Chama rotina para mostrar soma (envia o código 1 de
; r10 p/ end. de memória 10000 hex - Control Register)

halt
```

O exercício 3b (que irá consistir na adaptação do exercício 2b) – enviar o resultado da soma de 3 números para o terminal - fica para TPC...

Exercício 4

Adapte o exercício 3 de forma a obter do utilizador os valores a somar.

MIPS (MIPS64) – Assembler

- Obter valor inteiro ou de vírgula flutuante

- Memory Mapped I/O Area:

Memory Mapped I/O area

Addresses of CONTROL and DATA registers

CONTROL: .word32 0x10000

DATA: .word32 0x10008

Set CONTROL = 1, Set DATA to Unsigned Integer to be output

Set CONTROL = 2, Set DATA to Signed Integer to be output

Set CONTROL = 3, Set DATA to Floating Point to be output

Set CONTROL = 4, Set DATA to address of string to be output

Set CONTROL = 5, Set DATA+5 to x coordinate, DATA+4 to y coordinate, and DATA to RGB colour to be output

Set CONTROL = 6, Clears the terminal screen

Set CONTROL = 7, Clears the graphics screen

Set CONTROL = 8, read the DATA (either an integer or a floating-point) from the keyboard

Set CONTROL = 9, read one byte from DATA, no character echo.

- Para obter valores do utilizador na janela “terminal” (a partir do teclado) utiliza-se o **código 8**

Nota: Para mais informação sobre mapeamento de memória para E/S em MIPS64, consultar o ficheiro “**SC - MIPS64SimulatorIO.pdf**”

MIPS (MIPS64) – Assembler

- Obter valor inteiro ou de vírgula flutuante

- *Memory Mapped I/O Area:*

- **Obter valor inteiro** (ou de vírgula flutuante) **do teclado**

```
.data
CR: .word32 0x10000 ; Control Register tem o endereço 10000 hex
DR: .word32 0x10008 ; Data Register tem o endereço 10008 hex
; 0x é para indicar que é numero hexadecimal
```

```
.code
lwu r1, CR(r0) ; end. mem. do Control Register em r1
lwu r2, DR(r0) ; end. mem. do Data Register em r2
```

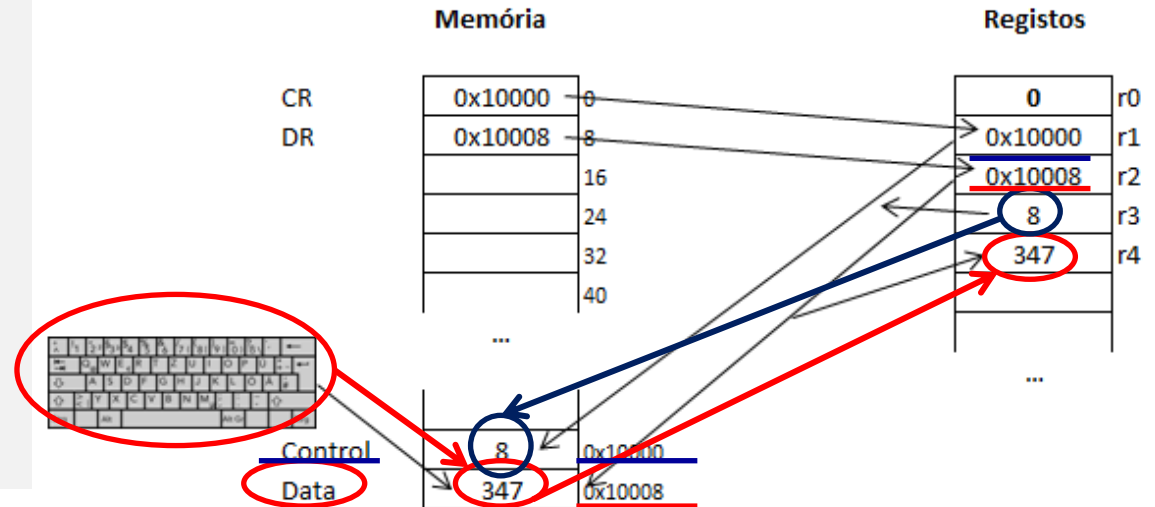
```
daddi r3,r0, 8 ; código 8 (código para obter dados do teclado) é colocado em r3
```

```
sd r3, (r1) ; chama rotina p/ obter valor do teclado (envia o código 8 de r3
; para end. de memória 10000 hex - Control Register)
```

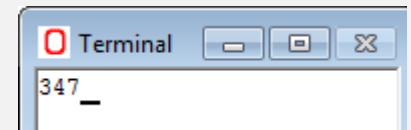
```
ld r4, (r2) ; valor digitado pelo teclado - e que fica guardado em Memória
; (Data Register) - é colocado em Registo (envia o valor 347 do
; endereço de memória 10008 hex p/ r4)
```

```
halt
```

Read an integer (or floating-point) from the keyboard (Ler r4)



Set CONTROL = 8, read the DATA (either an integer or a floating-point) from the keyboard



MIPS (MIPS64) - Assembler

- Obter inteiro (ou vírgula flutuante) de teclado (CODE 8) :
- Mostrar inteiro s/ sinal (CODE 1) no écran:

```
.data
CR: .word32 0x10000 ; Control Register tem o endereço 10000 hex
DR: .word32 0x10008 ; Data Register tem o endereço 10008 hex
```

```
.code
lwu r1, CR(r0)      ; end. mem. do Control Register em r1
lwu r2, DR(r0)      ; end. mem. do Data Register      em r2

daddi r3,r0, 8       ; cód. 8 (código p/ obter dados do teclado)

sd r3,(r1)           ; chama rotina p/ obter valor do teclado

ld r4,(r2)           ; valor digitado pelo teclado, que fica em
                    ; Memória (Data Register), colocado em Registro
halt
```

```
.code
lwu r1, CR(r0)      ; end. mem. do Control Register em r1
lwu r2, DR(r0)      ; end. mem. do Data Register      em r2

daddi r4,r0,20       ; valor (20) a mostrar na janela "terminal")

daddi r3,r0,1        ; cód. 1(código para mostrar inteiro sem sinal)

sd r4,(r2)           ; valor a mostrar é colocado no Data Register
sd r3,(r1)           ; chama rotina p/ mostrar valor
halt
```

Exercício 4a – Soma de 2 números

1. Utilizando o **notepad ++** abra novamente o ficheiro **“ex3a.s”**
2. Aceda a **“Ficheiro + Guardar Como”** e grave o ficheiro com o nome **“ex4a.s”**
3. Faça as alterações necessárias por forma a:
 - **Obter dados do utilizador pelo teclado:**
 - **CODE 8**
 - **Obter 1º número**
 - **CODE 8**
 - **Obter 2º número**
 - **Enviar o resultado da soma para o terminal (CODE 1) ... (já efetuado em ex3a)**

```

.data
A: .word 10
B: .word 8
C: .word 0
CR: .word32 0x10000 ; Control Register tem o endereço 10000
DR: .word32 0x10008 ; Data Register tem o endereço 10008

.code
lwu r1,CR(r0) ; end. mem. do Control Register em r1
lwu r2,DR(r0) ; end. mem. do Data Register em r2

daddi r10,r0,8 ; Código 8 (código para ler dados de teclado)

sd r10,(r1) ; Chama rotina para a introdução de dados de teclado
ld r4,(r2) ; Obtem 1º numero do teclado
ld r4,A(r0)

sd r10,(r1) ; Chama rotina para a introdução de dados do teclado
ld r5,(r2) ; Obtem 2º numero do teclado
ld r5,B(r0)

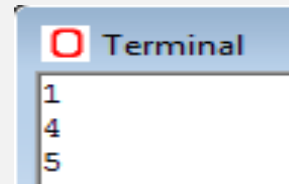
dadd r3,r4,r5

sd r3,C(r0) ; Colocar a soma que está em r3 na variável C
sd r3,(r2) ; Valor a mostrar é colocado no Data Register

daddi r10,r0,1 ; Código 1 (código p/ mostrar inteiro sem sinal) em r10
sd r10,(r1) ; Chama rotina para mostrar soma
halt
  
```

Exercício 4a – Soma de 2 números

- Exemplo da soma de 2 números:



Registers	Registers	Registers	Registers
R0= 0000000000000000	R0= 0000000000000000	R0= 0000000000000000	R0= 0000000000000000
R1= 0000000000001000	R1= 0000000000001000	R1= 0000000000001000	R1= 0000000000001000
R2= 0000000000001008	R2= 0000000000001008	R2= 0000000000001008	R2= 0000000000001008
R3= 0000000000000000	R3= 0000000000000000	R3= 0000000000000001	R3= 0000000000000005
R4= 0000000000000000	R4= 0000000000000001	R4= 0000000000000001	R4= 0000000000000001
R5= 0000000000000000	R5= 0000000000000004	R5= 0000000000000004	R5= 0000000000000004
R6= 0000000000000000	R6= 0000000000000000	R6= 0000000000000000	R6= 0000000000000000
R7= 0000000000000000	R7= 0000000000000000	R7= 0000000000000000	R7= 0000000000000000
R8= 0000000000000000	R8= 0000000000000000	R8= 0000000000000000	R8= 0000000000000000
R9= 0000000000000000	R9= 0000000000000000	R9= 0000000000000000	R9= 0000000000000000
R10= 0000000000000008	R10= 0000000000000008	R10= 0000000000000008	R10= 0000000000000001

- Exercício 4b (soma de 3 números a obter a partir do teclado) fica para TPC...

Consolidação de Conceitos – Variáveis vs. vetores em MIPS

- Em termos genéricos podemos ter:
 - **Variáveis** – estrutura que permite armazenar um único valor:
 - Por exemplo: “**A** = **1**” - a **variável A** armazena o valor **1**
 - **Vetores** – estrutura que permite armazenar um conjunto de valores, indexados por um índice que indica a posição:
 - Por exemplo: “**B**[**0**] = **20**” – a **primeira posição** do **vetor B** armazena o valor **20**
- No MIPS não há lugar à existência de “**A**”, pelo que a variável é tratada como um vetor mas de um só elemento. Para tal utiliza-se o formato “**Vetor(Índice)**”:
 - Sabendo que o registo **r0** tem sempre o valor “**0**”, então um vetor que seja referenciado como “**A**(**r0**)”, significa que estamos a indexar o **índice 0** do **vetor A**

Consolidação de Conceitos – Memory Mapped I/O Area

- Mostrar dados (valores inteiros) no écran (janela “terminal”):

Mostrar inteiro com sinal (**CODE 2**) no écran:

.data

CR: .word32 0x10000 ; 0x indica que é número hexadecimal – **Control Register** tem o **endereço 10000 hex**

DR: .word32 0x10008 ; 0x indica que é número hexadecimal – **Data Register** tem o **endereço 10008 hex**

.code

lwu r1, CR(r0) ; end. mem. do **Control Register** em r1

lwu r2, DR(r0) ; end. mem. do **Data Register** em r2

daddi r4,r0,37 ; **valor 37** (que é p/ ser mostrado na
; janela “terminal”) é colocado em r4

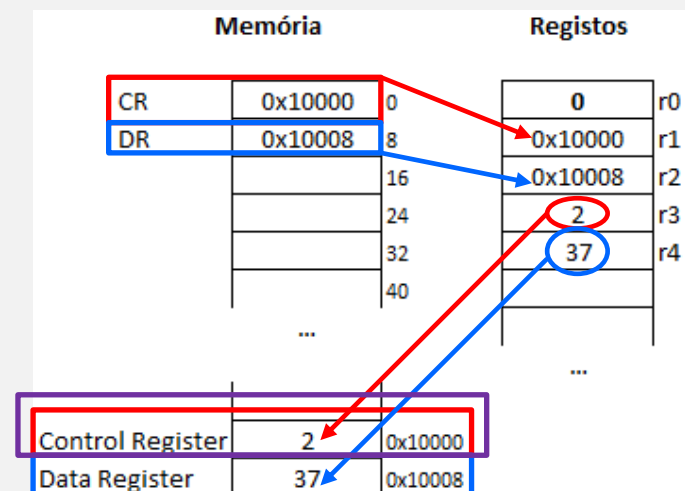
daddi r3,r0, 2 ; **código 2** (código para mostrar inteiro
; com sinal) é colocado em r3

sd r4,(r2) ; **valor a mostrar** é colocado no **Data**
; **Register** (envia o **valor 37** de r4 para o
; endereço de memória 10008 hex)

sd r3,(r1) ; **chama rotina p/ mostrar valor** (envia
; o **código 2** de r3 para o end. de mem.
; 10000 hex - **Control Register**)

halt

Quando é colocado um valor numérico na zona de memória denominada “**Control Register**”, “para tudo”, e é então invocada a rotina correspondente (neste caso é para mostrar dados numéricos – números inteiros com sinal)



Consolidação de Conceitos – Memory Mapped I/O Area

- Obtenção dados de teclado (digitados na janela “terminal”):

Obter inteiro (ou Vírgula Flutuante) de teclado (**CODE 8**):

.data

CR: .word32 0x10000 ; 0x indica que é número hexadecimal – **Control Register** tem o **endereço 10000 hex**

DR: .word32 0x10008 ; 0x indica que é número hexadecimal – **Data Register** tem o **endereço 10008 hex**

.code

lwu r1, CR(r0) ; end. mem. do **Control Register** em r1

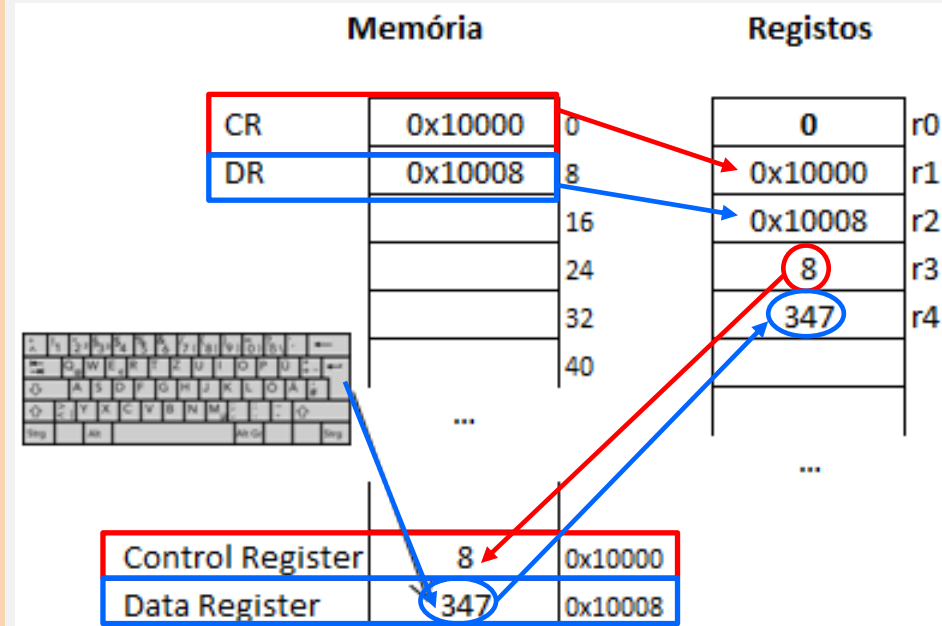
lwu r2, DR(r0) ; end. mem. do **Data Register** em r2

daddi r3,r0, **8** ; **código 8** (código para obter dados
; do teclado) é colocado em r3

sd r3,(r1) ; **chama rotina p/ obter valor** do teclado
; (envia o **código 8** de r3 para end. de
; memória 10000 hex - **Control Register**)

ld r4,(r2) ; **valor** digitado pelo teclado – e que fica
; guardado em **Memória (Data Register)**
; - é colocado em **Registro** (envia o **valor**
; **347** do end. de mem. 10008 hex p/ r4)

halt



Consolidação de Conceitos – Memory Mapped I/O Area

- Obtenção dados de Teclado vs. Mostrar dados no écran:

Obter inteiro (ou V.F.) de teclado (**CODE 8**):

Mostrar inteiro s/ sinal (**CODE 1**) no écran:

.data
CR: .word32 0x10000 ; **Control Register** tem o **endereço 10000**
DR: .word32 0x10008 ; **Data Register** tem o **endereço 10008**

.code
lwu r1, CR(r0) ; end. mem. do **Control Register** em r1
lwu r2, DR(r0) ; end. mem. do **Data Register** em r2

daddi r3,r0, **8** ; **código 8** (código para obter dados
; do teclado) é colocado em r3

sd r3,(**r1**) ; **chama rotina** p/ **obter valor** do teclado
; (envia o **código 8** de r3 para end. de
; memória 10000 hex - **Control Register**)

ld r4,(**r2**) ; **valor** digitado pelo teclado – e que fica
; guardado em **Memória (Data Register)**
; - é colocado em **Registo** (envia o **valor**
; **347** do end. de mem. 10008 hex p/ r4

halt

.code
lwu r1, CR(r0) ; end. mem. do **Control Register** em r1
lwu r2, DR(r0) ; end. mem. do **Data Register** em r2

daddi r4,r0,**20** ; **valor 20** (que é p/ ser mostrado na
; janela “terminal”) é colocado em r4

daddi r3,r0, **1** ; **código 1** (código para mostrar inteiro
; sem sinal) é colocado em r3

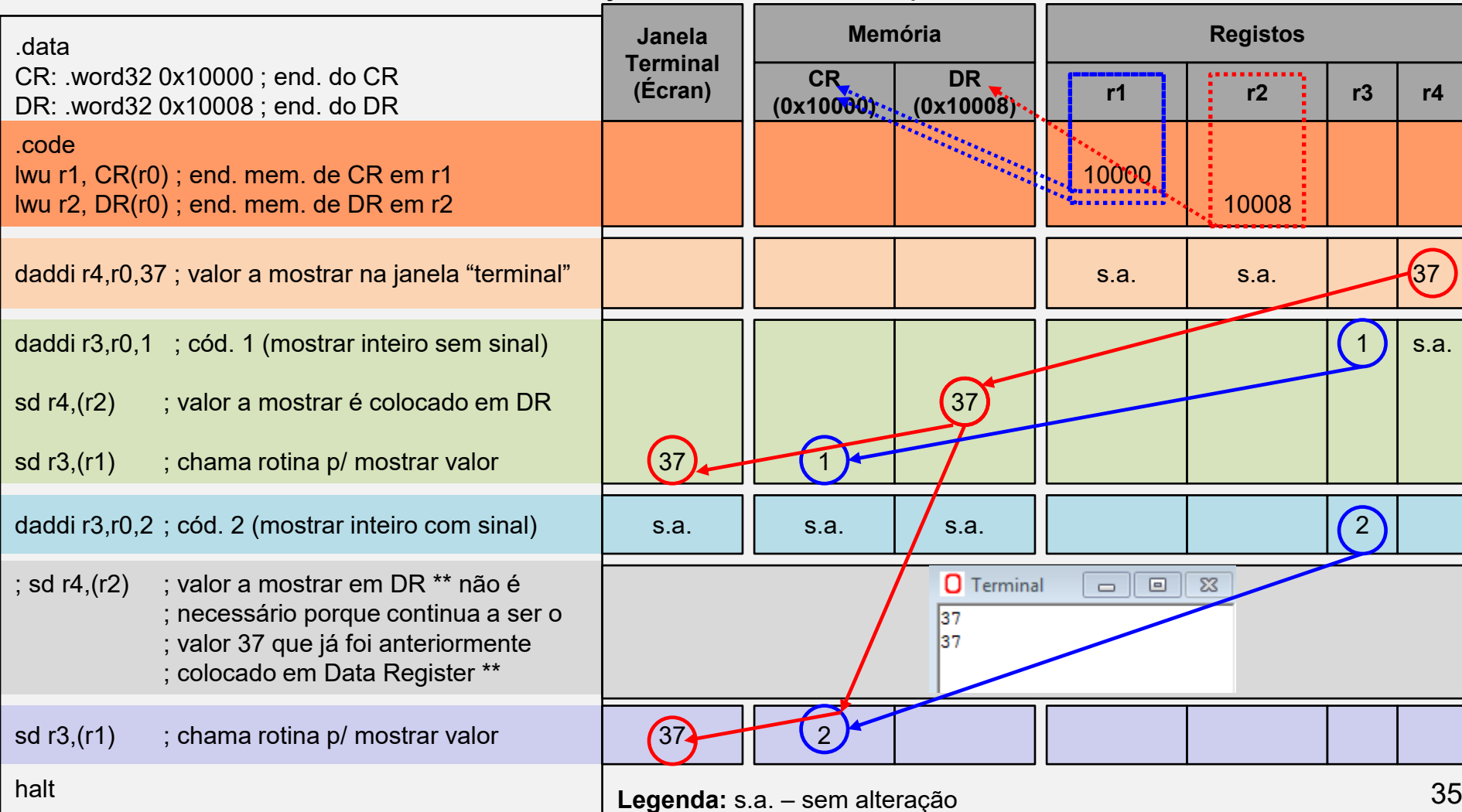
sd r4,(**r2**) ; **valor a mostrar** é colocado no **Data**
; **Register** (envia o **valor 20** de r4 p/ o
; o endereço de memória 10008 hex)

sd r3,(**r1**) ; **chama rotina** para **mostrar**
; **valor** (envia o **código 1** de r3
; para o endereço de memória
; 10000 hex - **Control Register**)

halt

Exercícios de Consolidação de Conceitos – Memory Mapped I/O Area

- Observando o código que se encontra à esquerda, tente compreender o comportamento de cada uma das estruturas MIPS e da janela “terminal” as quais estão identificadas à direita:

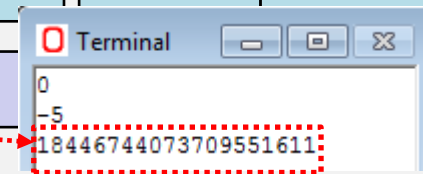


Exercícios de Consolidação de Conceitos – Memory Mapped I/O Area

- Observando o código que se encontra à esquerda, tente descrever o comportamento de cada uma das estruturas MIPS e da janela “terminal” as quais estão identificadas à direita:

	Janela Terminal (Écran)	Memória		Registos			
		CR (0x10000)	DR (0x10008)	r1	r2	r3	r4
.data CR: .word32 0x10000 ; end. do CR DR: .word32 0x10008 ; end. do DR				10000	10008		
.code lwu r1, CR(r0) ; end. mem. de CR em r1 lwu r2, DR(r0) ; end. mem. de DR em r2							
daddi r3,r0,1 ; cód. 1 (mostrar inteiro sem sinal) sd r3,(r1) ; chama rotina p/ mostrar valor - não ; mostra nada, pois não há nada em DR	0			s.a.	s.a.	1	
daddi r4,r0,-5 ; valor a mostrar na janela “terminal”							-5
daddi r3,r0,2 ; cód. 2 (mostrar inteiro com sinal) sd r4,(r2) ; valor a mostrar é colocado em DR sd r3,(r1) ; chama rotina p/ mostrar valor	-5	2	-5			2	s.a.
daddi r3,r0,1 ; cód. 1 (mostrar inteiro sem sinal)	s.a.	s.a.	s.a.			1	
sd r3,(r1) ; chama rotina p/ mostrar valor	??	1					
halt							

Legenda: s.a. – sem alteração

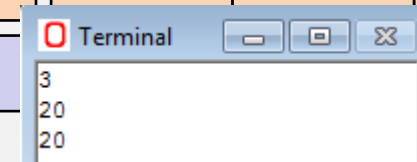


Exercícios de Consolidação de Conceitos – Memory Mapped I/O Area

- Observando o código que se encontra à esquerda, tente descrever o comportamento de cada uma das estruturas MIPS e da janela “terminal” as quais estão identificadas à direita:

	Janela Terminal (Écran)	Memória		Registos			
		CR (0x10000)	DR (0x10008)	r1	r2	r3	r4
.data CR: .word32 0x10000 ; end. do CR DR: .word32 0x10008 ; end. do DR				10000	10008		
.code lwu r1, CR(r0) ; end. mem. de CR em r1 lwu r2, DR(r0) ; end. mem. de DR em r2				s.a.	s.a.		3
daddi r4,r0,3 ; valor a mostrar na janela “terminal”						1	s.a.
daddi r3,r0,1 ; cód. 1 (mostrar inteiro sem sinal)			3				
sd r4,(r2) ; valor a mostrar é colocado em DR	3	1					
sd r3,(r1) ; chama rotina p/ mostrar valor							
daddi r3,r0,8 ; cód. 8 (obter valor)	s.a.	s.a.	s.a.			8	
sd r3,(r1) ; chama rotina p/ obter valor	20	8	20				20
ld r4,(r2) ; valor digitado pelo teclado (20)							
daddi r3,r0,1 ; cód. 1 (mostrar inteiro sem sinal)	s.a.	s.a.	s.a.			1	s.a.
sd r3,(r1) ; chama rotina p/ mostrar valor	20	1					
halt							

Legenda: s.a. – sem alteração



Créditos

Créditos

- Nuno Veiga, Rui Vasco Monteiro